ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO TIẾN ĐỘ DỰ ÁN 2

MÔN HỌC: ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI

SatiBot – Quản lý và tương tác với các tài nguyên AWS bằng ngôn ngữ tự nhiên

Giảng viên hướng dẫn:	TS. Hoàng Xuân Tùng
Nhóm thực hiện:	Nhóm 10
Sinh viên thực hiện:	Cao Anh Minh **(Trưởng nhóm)**
	Trịnh Thiên Quang
	Phạm Đức Hùng
	Nguyễn Đức Huy
Lớp môn học:	INT3319 1

Hà Nội, Tháng 10/2026

MỤC LỤC

1 TỔNG QUAN VỀ DỰ ÁN *(không thay đổi)* **1**

1.1 Giới thiệu chung về tính năng dự án 1

1.2 Giới thiệu người dùng của dự án 1

1.3 Cách thức và quy trình người dùng sử dụng sản phẩm để đạt mục đích . 2

1.3.1 Nhóm kịch bản và quy trình quản trị Amazon EC2 2

1.3.2 Nhóm kịch bản và quy trình quản trị Amazon S3 4

1.3.3 Nhóm kịch bản và quy trình quản trị AWS IAM 5

1.3.4 Nhóm kịch bản và quy trình quản trị AWS Lambda 6

2 GIẢI PHÁP THỰC HIỆN CHÍNH THỨC **8**

2.1 Kiến trúc tính năng của hệ thống 8

2.2 Kiến trúc triển khai của hệ thống 9

2.2.1 Sơ đồ kiến trúc triển khai hệ thống 10

2.3 Đặc tả chi tiết cần thiết trong giải pháp của dự án 11

2.3.1 Đặc tả cấu hình dịch vụ Bảo mật và Điều hướng 11

2.3.2 Giao diện và giao thức tương tác giữa các components 13

2.3.3 Cơ sở dữ liệu hệ thống 13

2.3.4 CI/CD 13

2.3.5 Phân quyền cho các hàm Lambda 15

2.3.6 Tích hợp LLM qua Gemini API 16

2.3.7 Endpoint tiếp nhận hội thoại và trích xuất ý định (POST /chat) 17

2.3.8 Endpoint xác nhận và thực thi tài nguyên (POST /confirm) . . 19

3 KẾ HOẠCH TRIỂN KHAI DỰ ÁN **21**

3.1 Kế hoạch triển khai theo từng giai đoạn (05 tuần) 21

3.2 Kế hoạch hành động chi tiết theo từng Sprint 22

1. TỔNG QUAN VỀ DỰ ÁN *(không thay đổi)*

1.1 Giới thiệu chung về tính năng dự án

Hiện nay, Amazon Web Services (AWS) là một trong những nền tảng điện toán đám mây phổ biến nhất phục vụ học tập, nghiên cứu và triển khai ứng dụng. Tuy nhiên, việc khởi tạo và quản trị tài nguyên thông qua giao diện web (AWS Management Console) đòi hỏi nhiều thao tác thủ công, điều hướng qua nhiều màn hình cấu hình phức tạp và rất dễ nhầm lẫn. Trong khi đó, việc sử dụng giao diện dòng lệnh (AWS CLI) hoặc viết tập lệnh tự động lại yêu cầu ghi nhớ chính xác cú pháp câu lệnh, cấu trúc tệp cấu hình JSON và các tham số kỹ thuật. Đối với sinh viên, lập trình viên mới tiếp cận điện toán đám mây hoặc người quản lý kỹ thuật, các rào cản này làm chậm quá trình vận hành, đồng thời tiềm ẩn nguy cơ cấu hình sai gây mất an toàn thông tin hoặc phát sinh chi phí ngoài ý muốn.

Nhằm giải quyết vấn đề trên, nhóm thực hiện đề tài xây dựng SatiBot – ứng dụng ChatOps hỗ trợ quản lý, cấu hình và giám sát tài nguyên AWS thông qua câu lệnh bằng ngôn ngữ tự nhiên tiếng Việt. Bằng cách kết hợp mô hình ngôn ngữ lớn (LLM) hỗ trợ cơ chế *Function Calling* với thư viện AWS SDK (Boto3), SatiBot hoạt động như một trợ lý hỗ trợ thao tác hạ tầng. Người dùng chỉ cần gửi yêu cầu bằng tiếng Việt qua giao diện chat; hệ thống sẽ phân tích ý định, kiểm tra tính hợp lệ và độ an toàn của câu lệnh, sau đó gọi các API tương ứng của AWS và trả về kết quả thực thi kèm theo thông tin chi tiết, cảnh báo rủi ro hoặc hướng dẫn xử lý tiếp theo.

1.2 Giới thiệu người dùng của dự án

Hệ thống SatiBot hướng tới 03 nhóm người dùng chính với nhu cầu sử dụng thực tế như sau:

- **Lập trình viên và kỹ sư vận hành (Developers / DevOps Engineers):** Cần khởi tạo nhanh môi trường thử nghiệm, cấu hình máy chủ, cập nhật mã nguồn hàm serverless hoặc xem log hệ thống trong quá trình phát triển ứng dụng. SatiBot giúp nhóm này thực hiện các tác vụ lặp đi lặp lại nhanh hơn ngay trên khung chat mà không cần chuyển đổi qua lại giữa nhiều trang trên AWS Console.
- **Sinh viên ngành Công nghệ thông tin / Điện toán đám mây:** Cần môi trường thực hành trực quan cho các môn học về mạng và hạ tầng đám mây. Hệ thống cung cấp cơ chế kiểm tra an toàn (guardrails), giúp sinh viên tránh các lỗi phổ biến như mở cổng mạng công khai nguy hiểm, quên tắt máy ảo hoặc chọn nhầm các gói cấu hình vượt quá phạm vi gói miễn phí (AWS Free Tier).
- **Quản lý kỹ thuật và chủ dự án (Tech Leads / Project Managers):** Cần theo dõi

nhanh tình trạng hoạt động của tài nguyên, kiểm tra phân quyền (IAM Audit) và kiểm soát chi phí sử dụng hàng tháng (FinOps) bằng ngôn ngữ tự nhiên mà không cần thành thạo cú pháp dòng lệnh AWS CLI.

1.3 Cách thức và quy trình người dùng sử dụng sản phẩm để đạt mục đích

Để đáp ứng nhu cầu quản trị hạ tầng cơ bản, SatiBot hỗ trợ 04 dịch vụ chính của AWS gồm: Amazon EC2, Amazon S3, AWS IAM và AWS Lambda. Dưới đây là chi tiết về mục đích và quy trình tương tác thực tế của người dùng đối với từng nhóm tính năng của 04 dịch vụ AWS nòng cốt.

1.3.1 Nhóm kịch bản và quy trình quản trị Amazon EC2

Dịch vụ máy chủ ảo (Amazon Elastic Compute Cloud - EC2) là hạ tầng phục vụ việc triển khai và chạy ứng dụng. SatiBot hỗ trợ 05 quy trình quản lý trạng thái và cấu hình máy ảo như sau:

1. **Khởi tạo máy chủ chuẩn hóa và xử lý trùng lặp tên:**
   - *Mục đích:* Tạo nhanh máy ảo để thử nghiệm hoặc triển khai website mà không phải cấu hình thủ công nhiều bước, đồng thời xử lý trường hợp tên máy bị trùng.
   - *Quy trình người dùng:*
     – Nhắn yêu cầu tạo máy kèm mục đích sử dụng (ví dụ: ”Tạo cho tôi một máy chủ Ubuntu để chạy web demo, mở cổng SSH và HTTP”).
     – Nếu tên máy đã tồn tại, hệ thống đưa ra 3 lựa chọn: tiếp tục sử dụng máy cũ, đổi tên máy mới hoặc xóa máy cũ để tạo lại.
     – Nhận kết quả thành công gồm: Địa chỉ IP máy chủ, link tải file khóa SSH (.pem) và câu lệnh kết nối mẫu.
2. **Kiểm soát chi phí và xử lý vượt hạn mức tài nguyên (Cost Guardrails & Quota):**
   - *Mục đích:* Tránh việc chọn nhầm máy cấu hình cao gây phát sinh chi phí lớn, đồng thời hướng dẫn cách xử lý khi tài khoản chạm ngưỡng hạn mức tạo máy ảo (vCPU quota).
   - *Quy trình người dùng:*
     – Nhắn yêu cầu loại máy mong muốn (ví dụ: *”Tạo máy ảo c5.4xlarge 16 vCPU”* hoặc *”Tạo thêm 1 máy ảo Ubuntu”*).
     – Xem phản hồi từ hệ thống:
       ∗ *Nếu yêu cầu máy cấu hình cao, chi phí lớn:* Hệ thống cảnh báo chi phí ước tính hàng tháng, tạm dừng thao tác và yêu cầu người dùng xác nhận nếu vẫn muốn tiếp tục khởi tạo.
       ∗ *Nếu tài khoản chạm giới hạn vCPU:* Hệ thống giải thích nguyên nhân và liệt

kê danh sách các máy ảo đang chạy để người dùng cân nhắc tắt bớt máy không dùng.

– Người dùng chọn phương án: chuyển về gói miễn phí (Free Tier), tắt bớt máy cũ hoặc gửi yêu cầu nâng hạn mức lên AWS.

3. **Tra cứu và điều khiển vòng đời máy chủ (Bật / Tắt / Khởi động lại / Xóa):**
   - *Mục đích:* Theo dõi danh sách máy ảo và điều khiển trạng thái bật/tắt kịp thời nhằm tránh lãng phí tài nguyên và kiểm soát chi phí vận hành.
   - *Quy trình người dùng:*
     – Nhắn câu lệnh tra cứu hoặc điều khiển (ví dụ: *"Liệt kê các máy ảo đang chạy"*, *"Tắt máy ảo web-demo"*, *"Khởi động lại máy i-0abc123"*, *"Xóa máy ảo staging"*).
     – Xem bảng thông tin gồm ID, tên máy, trạng thái, IP công khai và thời gian chạy; nhận thông báo trạng thái cập nhật (kèm IP mới khi bật lại máy).
     – Với thao tác xóa: Hệ thống cảnh báo mất dữ liệu và yêu cầu người dùng gõ xác nhận trước khi thực hiện xóa máy.
4. **Giám sát hiệu năng và phát hiện máy chủ lãng phí (FinOps & CloudWatch):**
   - *Mục đích:* Theo dõi mức độ sử dụng tài nguyên để tắt các máy nhàn rỗi nhằm tiết kiệm chi phí, hoặc nhận biết máy bị quá tải để kịp thời nâng cấp.
   - *Quy trình người dùng:*
     – Nhắn câu hỏi kiểm tra (ví dụ: *"Kiểm tra CPU máy i-0abc123 trong 24 giờ qua"*).
     – Nhận số liệu đánh giá chi tiết:
       ∗ *Cảnh báo lãng phí:* Nếu máy chạy không tải kéo dài, hệ thống đưa ra gợi ý tắt máy để tiết kiệm chi phí.
       ∗ *Cảnh báo nghẽn tải:* Nếu mức sử dụng CPU liên tục ở mức cao, hệ thống cảnh báo quá tải và gợi ý nâng cấp loại máy.
     – Người dùng nhắn đồng ý nếu muốn hệ thống tự động xử lý theo khuyến nghị.
5. **Thiết lập tường lửa an toàn và hỗ trợ sự cố kết nối (SecOps):**
   - *Mục đích:* Bảo vệ máy chủ khỏi nguy cơ bị quét dò mật khẩu từ Internet (brute-force attack) và hỗ trợ chẩn đoán nhanh nguyên nhân khi không kết nối SSH được vào máy ảo.
   - *Quy trình người dùng:*
     – Người dùng gửi yêu cầu mở cổng hoặc hỏi nguyên nhân sự cố (ví dụ: *"Mở port 22 cho mọi người truy cập"* hoặc *"Tôi không SSH được vào máy ảo i-0abc123, sao vậy?"*).
     – *Kiểm tra an toàn tường lửa:* Hệ thống từ chối mở port 22 cho toàn bộ dải mạng công khai (0.0.0.0/0), giải thích nguy cơ bị tấn công dò quét và gợi ý chỉ mở riêng

cho địa chỉ IP hiện tại của người dùng.

- *Chẩn đoán kết nối:* Nếu phát hiện người dùng mất file khóa SSH (.pem), hệ thống giải thích nguyên nhân và hướng dẫn sử dụng các kênh truy cập thay thế (như AWS Systems Manager Session Manager hoặc EC2 Instance Connect).
- Người dùng phản hồi đồng ý để áp dụng cấu hình an toàn được đề xuất.

1.3.2 Nhóm kịch bản và quy trình quản trị Amazon S3

Dịch vụ lưu trữ đối tượng (Amazon Simple Storage Service - S3) dùng để lưu trữ tập tin, bản sao lưu và dữ liệu tĩnh của hệ thống. SatiBot hỗ trợ 06 quy trình quản lý S3 như sau:

1. **Khởi tạo thùng chứa (S3 Bucket) an toàn và chuẩn hóa:**
   - *Mục đích:* Tạo kho lưu trữ đối tượng được thiết lập ở chế độ riêng tư theo mặc định và tuân thủ quy tắc đặt tên của AWS.
   - *Quy trình người dùng:* Nhắn yêu cầu tạo kèm tên mong muốn (ví dụ: *"Tạo bucket tailieu-backup"*). Nếu tên bị trùng hoặc sai quy chuẩn đặt tên, xem hướng dẫn từ hệ thống và nhắn lại tên mới. Nhận thông báo bucket đã được tạo thành công ở chế độ riêng tư.
2. **Quét và khóa rủi ro lộ dữ liệu ra Internet (Security Scan & Remediation):**
   - *Mục đích:* Rà soát và ngăn chặn kịp thời tình trạng lộ dữ liệu do vô tình mở quyền truy cập công khai (Public Access) trên bucket.
   - *Quy trình người dùng:* Nhắn yêu cầu kiểm tra (ví dụ: *"Kiểm tra xem có bucket nào đang bị lộ ra ngoài Internet không?"*). Hệ thống quét chính sách truy cập của toàn bộ bucket trong tài khoản và trả về danh sách các bucket đang có nguy cơ. Người dùng xác nhận (ví dụ: *"Khóa lại giùm tôi"*) để hệ thống bật tính năng Block Public Access ngay lập tức.
3. **Xóa bucket S3 an toàn có kiểm soát (chống xóa nhầm dữ liệu):**
   - *Mục đích:* Dọn dẹp bucket cũ mà không lo ngại nguy cơ xóa nhầm dữ liệu quan trọng đang lưu bên trong.
   - *Quy trình người dùng:* Người dùng nhắn yêu cầu xóa (ví dụ: *"Xóa bucket du-an-2025"*). Hệ thống kiểm tra và thông báo số lượng tệp tin còn tồn bên trong. Người dùng phải nhập xác nhận rõ ràng; khi đó hệ thống mới tiến hành dọn sạch các đối tượng và xóa bucket.
4. **Tối ưu chi phí lưu trữ lâu dài (FinOps Lifecycle Rules):**
   - *Mục đích:* Giảm thiểu chi phí lưu trữ bằng cách tự động chuyển các tệp cũ, ít sử dụng sang các tầng lưu trữ giá rẻ hơn (S3 Standard-IA, S3 Glacier).
   - *Quy trình người dùng:* Người dùng nhắn yêu cầu chuyển tầng lưu trữ kèm mốc thời

gian (ví dụ: *"Tự chuyển file trong bucket sang kho lưu trữ giá rẻ sau 30 ngày"*). Hệ thống thiết lập quy tắc vòng đời (Lifecycle Configuration Rule) trên S3 và gửi thông báo xác nhận chính sách đã được áp dụng.

5. **Chia sẻ tệp tin an toàn có thời hạn (Presigned URL):**
   - *Mục đích:* Cung cấp liên kết tải tệp tin an toàn cho người ngoài trong khoảng thời gian nhất định mà không cần mở public dữ liệu hay cấp quyền tài khoản AWS.
   - *Quy trình người dùng:* Người dùng nhắn tên tệp và thời hạn chia sẻ (ví dụ: *"Tạo link tải file hop-dong-2026.pdf trong 15 phút"*). Hệ thống tạo đường dẫn có chữ ký xác thực (Presigned URL) với thời gian hết hạn (TTL) 15 phút và gửi liên kết trực tiếp trên khung chat.
6. **Tra cứu dung lượng và tình trạng kho lưu trữ (Storage Analytics):**
   - *Mục đích:* Nắm bắt nhanh số lượng tệp tin, dung lượng sử dụng thực tế và dự toán chi phí lưu trữ của từng bucket.
   - *Quy trình người dùng:* Người dùng đặt câu hỏi tra cứu (ví dụ: *"Bucket tailieu-duan có bao nhiêu file và chiếm bao nhiêu GB?"*). Hệ thống truy vấn CloudWatch Metrics và S3 API để thống kê chi tiết: số lượng objects, dung lượng tổng (GB), tầng lưu trữ và chi phí ước tính, hiển thị dưới dạng bảng rõ ràng.

1.3.3 Nhóm kịch bản và quy trình quản trị AWS IAM

Dịch vụ quản lý định danh và quyền truy cập (AWS Identity and Access Management - IAM) là trung tâm kiểm soát an ninh của hạ tầng AWS. SatiBot cung cấp 05 quy trình quản trị IAM an toàn, tuân thủ nguyên tắc đặc quyền tối thiểu (Least Privilege):

1. **Tra cứu vai trò (Role) và chính sách quyền hạn (Policy):**
   - *Mục đích:* Xem nhanh danh sách hoặc kiểm tra chi tiết các quyền hạn đang được gán cho nhân sự hoặc dịch vụ để phục vụ công tác kiểm toán an ninh.
   - *Quy trình người dùng:* Người dùng nhắn câu hỏi tra cứu (ví dụ: *"Kiểm tra xem Role lambda-s3-role đang có những quyền gì?"* hoặc *"Liệt kê các Role dành cho dịch vụ EC2"*). Hệ thống phân tích tài liệu chính sách JSON và hiển thị bảng tóm tắt quyền hạn (tên quyền, dịch vụ được phép truy cập, phạm vi đọc/ghi) ngay trên khung chat.
2. **Khởi tạo vai trò (Role) hoặc chính sách (Policy) mới theo nguyên tắc tối thiểu:**
   - *Mục đích:* Tạo mới đối tượng phân quyền cho máy chủ, ứng dụng hoặc nhân sự mà không phải tự tay viết các đoạn mã JSON chính sách phức tạp và dễ nhầm lẫn.
   - *Quy trình người dùng:* Người dùng nhắn yêu cầu tạo kèm mục đích (ví dụ: *"Tạo một Role cho máy ảo EC2 chỉ được phép đọc dữ liệu từ S3"* hoặc *"Tạo Policy cho phép đọc log CloudWatch"*). Hệ thống đề xuất cấu trúc quyền hạn phù hợp (chỉ cấp quyền đọc dữ liệu, không cấp thừa quyền ghi/xóa). Sau khi người dùng xác nhận,

hệ thống gọi API tạo đối tượng và trả về ARN của Role/Policy.

3. **Cấp hoặc điều chỉnh quyền truy cập:**
   - *Mục đích:* Bổ sung hoặc nới rộng quyền hạn cho nhân sự/dịch vụ khi phát sinh công việc mới trong dự án.
   - *Quy trình người dùng:* Người dùng nhắn yêu cầu gán thêm quyền (ví dụ: *"Cấp thêm quyền đọc DynamoDB cho Role app-backend"*). Hệ thống đối soát đối tượng nhận quyền và chính sách tương ứng, yêu cầu người dùng xác nhận và trả về thông báo quyền truy cập mới đã được áp dụng thành công.
4. **Thu hồi quyền truy cập (Least Privilege Enforcement):**
   - *Mục đích:* Gỡ bỏ các quyền hạn không còn sử dụng hoặc thu hồi các quyền nguy hiểm (như quyền AdministratorAccess cấp thừa cho tài khoản thực tập sinh) nhằm hạn chế rủi ro an toàn thông tin.
   - *Quy trình người dùng:* Người dùng nhắn yêu cầu gỡ bỏ quyền (ví dụ: *"Gỡ quyền AdministratorAccess khỏi tài khoản dev-intern"*). Hệ thống hiển thị thông báo phạm vi quyền chuẩn bị gỡ bỏ; sau khi người dùng xác nhận, hệ thống gỡ bỏ chính sách và thông báo kết quả.
5. **Xóa đối tượng IAM an toàn:**
   - *Mục đích:* Dọn dẹp dứt điểm các Role, Policy hoặc User cũ không còn sử dụng để tránh dư thừa tài nguyên và loại bỏ lỗ hổng bảo mật tiềm tàng.
   - *Quy trình người dùng:* Người dùng gửi yêu cầu xóa (ví dụ: *"Xóa Role test-migration cũ giùm tôi"*). Hệ thống kiểm tra các quan hệ phụ thuộc (nếu Role đang gắn với máy ảo EC2 đang chạy, hệ thống sẽ cảnh báo nguy cơ làm dừng ứng dụng). Người dùng xác nhận lần cuối để hệ thống tự động tháo gỡ các liên kết an toàn trước khi xóa bỏ đối tượng IAM.

1.3.4 Nhóm kịch bản và quy trình quản trị AWS Lambda

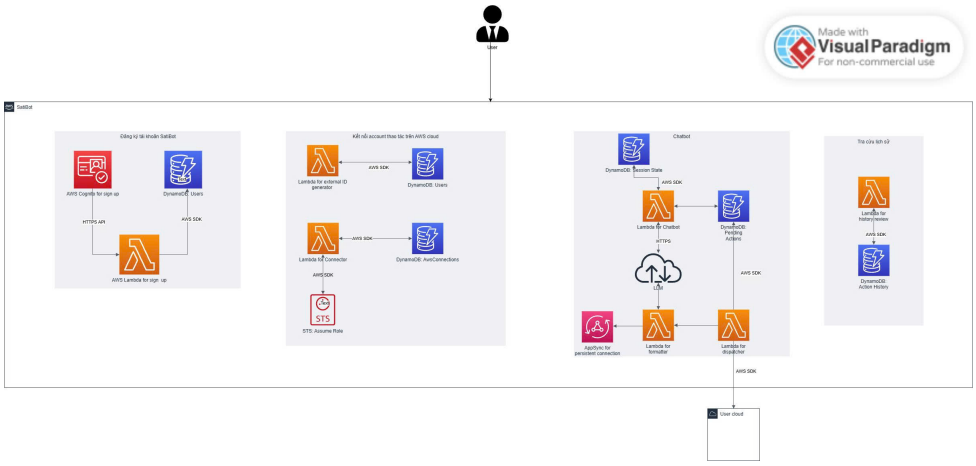
Dịch vụ điện toán không máy chủ (AWS Lambda) cho phép chạy mã nguồn theo sự kiện. SatiBot hỗ trợ 05 quy trình quản trị và kiểm thử hàm serverless trực tiếp:

1. **Tra cứu danh mục và kiểm tra cấu hình hàm (Inventory & Config Audit):**
   - *Mục đích:* Nắm bắt nhanh toàn bộ các hàm Lambda đang có trong tài khoản cùng các thông số vận hành cốt lõi (ngôn ngữ runtime, bộ nhớ RAM, thời gian timeout) phục vụ công tác kiểm toán tài nguyên.
   - *Quy trình người dùng:* Người dùng gửi câu hỏi (ví dụ: *"Liệt kê các hàm Lambda trong tài khoản"* hoặc *"Hàm image-processor đang chạy ngôn ngữ gì và cấp bao nhiêu RAM?"*). Hệ thống hiển thị bảng tổng hợp gồm Tên hàm, Môi trường thực thi (Runtime), Dung lượng bộ nhớ (Memory) và Thời gian chờ tối đa (Timeout).

2. **Chạy thử nghiệm hàm trực tiếp qua Chat (Direct Invocation):**
   - *Mục đích:* Kích hoạt và kiểm tra nhanh kết quả trả về của hàm với dữ liệu thử nghiệm ngay trên khung chat mà không cần đợi sự kiện bên ngoài hay cấu hình test event trên Console.
   - *Quy trình người dùng:* Người dùng nhắn lệnh chạy kèm dữ liệu đầu vào (ví dụ: *"Chạy thử hàm calculate-price với dữ liệu giá 100 và thuế 10"*). Hệ thống đóng gói dữ liệu JSON, gọi API 'invoke', đo thời gian thực thi và trả về kết quả (Payload phản hồi, mã HTTP status, thời gian chạy tính bằng mili-giây) trực tiếp trên khung chat.
3. **Điều chỉnh cấu hình bộ nhớ và thời gian chờ của hàm Lambda:**
   - *Mục đích:* Nâng cấp cấu hình tức thời cho các hàm đang chạy chậm hoặc có nguy cơ sập do thiếu bộ nhớ hay hết thời gian chờ mà không phải can thiệp mã nguồn hay mở Console.
   - *Quy trình người dùng:* Người dùng ra lệnh điều chỉnh thông số (ví dụ: *"Tăng thời gian timeout của hàm backup-data lên 30 giây"* hoặc *"Nâng RAM của hàm image-processor lên 512 MB"*). Hệ thống hiển thị bảng đối soát thông số cũ và thông số mới. Sau khi người dùng xác nhận, hệ thống gọi API cập nhật cấu hình và thông báo thành công.
4. **Cấp phát nhanh liên kết HTTP công khai (Function URL Generation):**
   - *Mục đích:* Tạo nhanh một đường dẫn Endpoint HTTPS công khai (Function URL) để người dùng hoặc webhook bên ngoài có thể gọi trực tiếp hàm Lambda mà không mất công cấu hình API Gateway phức tạp.
   - *Quy trình người dùng:* Người dùng nhắn: *"Tạo link Function URL công khai cho hàm image-api"*. Hệ thống tự động cấu hình Function URL, thiết lập quyền truy cập và gửi lại đường link HTTPS dạng https://<url-id>.lambda-url.<region>.on.aws/ trực tiếp trên khung chat để người dùng kiểm chứng kết quả thực thi của hàm.
5. **Tạm dừng hoạt động hàm khẩn cấp (Emergency Kill-Switch):**
   - *Mục đích:* Vô hiệu hóa ngay lập tức một hàm đang bị lặp lỗi logic gây tiêu hao ngân sách hoặc cần ngắt kết nối gấp để bảo trì hệ thống mà không cần xóa bỏ mã nguồn của hàm.
   - *Quy trình người dùng:* Người dùng nhắn lệnh ngắt khẩn cấp (ví dụ: *"Khóa tạm thời hàm order-service để bảo trì hệ thống"*). Hệ thống cảnh báo về việc hàm sẽ từ chối mọi yêu cầu mới (trả về lỗi HTTP 429 Too Many Requests) và đề nghị người dùng xác nhận. Sau khi nhận xác nhận, hệ thống đặt mức thực thi đồng thời về 0 (`Reserved Concurrency = 0`), đưa hàm vào trạng thái tạm ngừng hoạt động an toàn. Người dùng có thể yêu cầu mở lại mức concurrency bất cứ lúc nào.

2. GIẢI PHÁP THỰC HIỆN CHÍNH THỨC

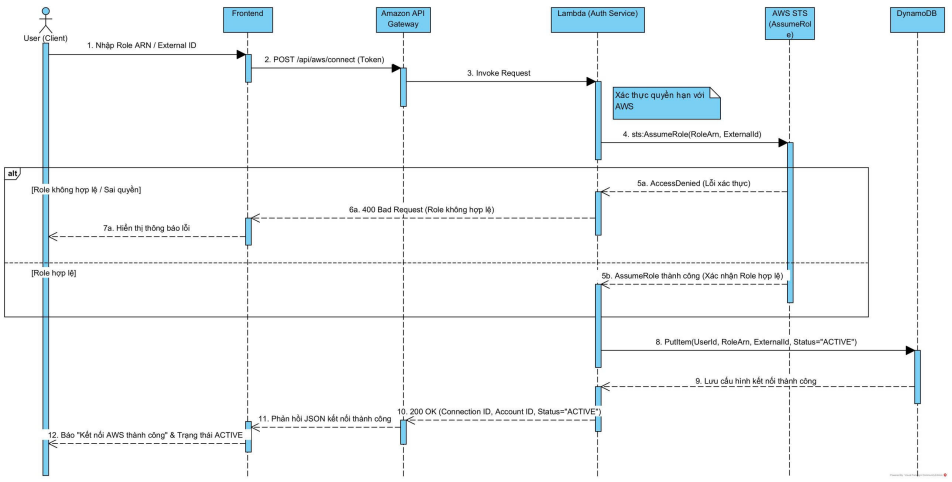
2.1 Kiến trúc tính năng của hệ thống



Hình 1. Sơ đồ kiến trúc tính năng và luồng xử lý nghiệp vụ của SatiBot

Các tính năng hệ thống cung cấp bao gồm:

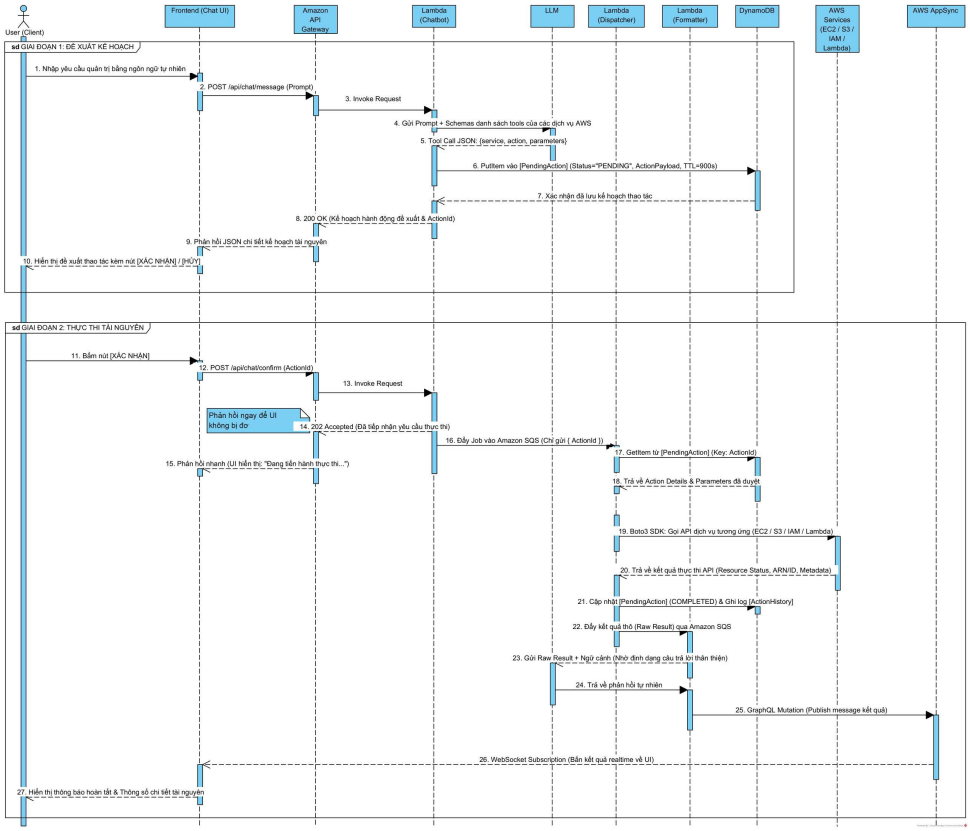
1. **Đăng ký và xác thực người dùng:** Amazon Cognito xử lý đăng ký, đăng nhập và cấp JWT để xác thực các yêu cầu nghiệp vụ. Sau khi đăng ký, Lambda tạo hồ sơ người dùng trong bảng `Users` của DynamoDB.
2. **Kết nối tài khoản AWS:** SatiBot tạo `ExternalId`; người dùng dùng mã này và hướng dẫn của hệ thống để tạo IAM Role trong tài khoản AWS của mình, sau đó gửi `RoleArn` về SatiBot. Connector Lambda gọi STS `AssumeRole` để kiểm tra khả năng ủy quyền. Nếu thành công, hệ thống lưu thông tin kết nối và trạng thái trong bảng `AwsConnections`.



Hình 2. Sơ đồ tuần tự quy trình kết nối và xác thực tài khoản AWS mục tiêu

3. **Quản trị tài nguyên qua Chatbot:** Chatbot Lambda nhận câu lệnh, lấy ngữ cảnh hội thoại từ `SessionState` và gửi yêu cầu cùng Function Calling Schema đến mô hình

LLM. Kết quả được lưu thành hành động chờ xác nhận trong `PendingAction`. Khi người dùng xác nhận, Chatbot Lambda chuyển ActionID vào SQS cho Dispatcher Lambda; Dispatcher Lambda dùng STS và Boto3 để gọi API của EC2, S3, IAM hoặc Lambda trên AWS của người dùng. Kết quả được lưu vào `ActionHistory` và gửi về giao diện qua AppSync.



Hình 3. Sơ đồ tuần tự quy trình điều phối và thực thi tác vụ ChatOps

4. **Tra cứu lịch sử thao tác:** History Lambda truy vấn bảng `ActionHistory` theo người dùng và các điều kiện tra cứu, sau đó trả về thời gian, hành động, tài nguyên đích và trạng thái thực thi để hiển thị trên giao diện.

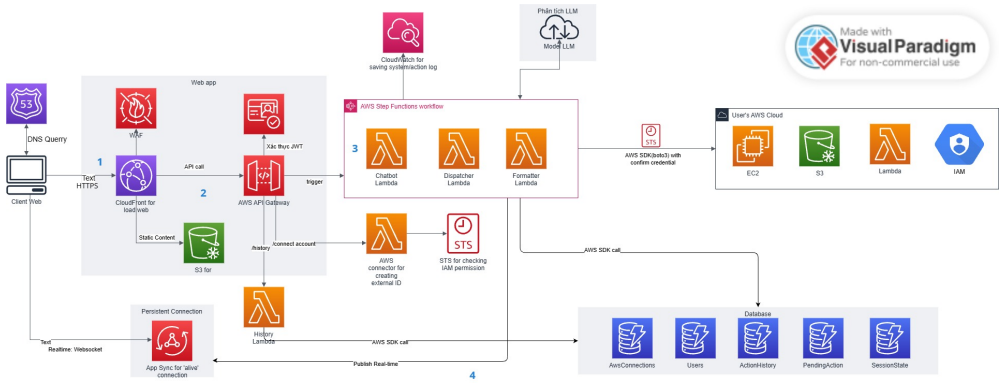
2.2 Kiến trúc triển khai của hệ thống

SatiBot sử dụng các dịch vụ AWS được quản lý để triển khai giao diện, xác thực, xử lý nghiệp vụ và lưu trữ dữ liệu.

Bảng 1. Các thành phần và dịch vụ trong kiến trúc triển khai hệ thống SatiBot

Thành phần / Dịch vụ	Vai trò trong kiến trúc
AWS WAF	Tường lửa ứng dụng web, gắn vào CloudFront để lọc request.
Amazon Route 53	Quản lý tên miền (DNS), điều hướng người dùng đến CloudFront.
React, Amazon S3 và CloudFront	Cung cấp giao diện ChatOps; S3 lưu các tệp tĩnh và CloudFront phân phối giao diện qua HTTPS.
Amazon Cognito	Quản lý đăng ký, đăng nhập và cấp JWT để xác thực người dùng.
AWS STS	Cấp quyền truy cập tạm thời (Temporary Credentials).
Amazon API Gateway	Tiếp nhận API từ frontend, xác thực JWT và định tuyến yêu cầu chat, xác nhận thao tác hoặc tra cứu lịch sử.
AWS Step Functions	Điều phối luồng thực thi sau khi người dùng xác nhận, gồm ChatBot, Dispatcher và Formatter Lambda.
AWS Lambda	Thực hiện các nghiệp vụ: Chatbot, Dispatcher, Formatter, Connector và History; Connector đảm nhiệm tạo ExternalId và kiểm tra kết nối AWS.
Mô hình LLM	Phân tích câu lệnh tiếng Việt và trả về ý định, tham số theo Function Calling Schema; không trực tiếp gọi API AWS.
AWS STS và Boto3	STS cấp thông tin xác thực tạm thời cho tài khoản đích; Boto3 dùng thông tin này để gọi API tài nguyên.
Amazon DynamoDB	Lưu hồ sơ người dùng, kết nối AWS, lịch sử hội thoại, hành động chờ xác nhận và lịch sử thao tác.
AWS AppSync	Phát kết quả qua GraphQL Subscriptions để giao diện cập nhật mà không cần polling liên tục.
Amazon CloudWatch	Lưu log thực thi và hỗ trợ theo dõi, chẩn đoán lỗi.
CI/CD Pipeline (GitHub, CodePipeline, CodeBuild)	Tự động hóa quy trình build, test và deploy.

2.2.1 Sơ đồ kiến trúc triển khai hệ thống



Hình 4. Sơ đồ kiến trúc triển khai tổng thể hệ thống SatiBot

Luồng hoạt động và giao tiếp giữa các thành phần gồm bốn bước:

1. **Tiếp nhận yêu cầu:** Người dùng truy cập ứng dụng web thông qua giao thức HTTPS. Yêu cầu được gửi đến AWS Route53 và trả về IP của CloudFront. Request sẽ đi qua AWS WAF để chặn các request xấu (như chặn SQL Injection, Cross-Site Scripting

(XSS), hoặc giới hạn tần suất request để chống DDoS). Sau đó, Amazon CloudFront là nơi phân phối nội dung tĩnh (HTML, CSS, JS) từ Amazon S3. Các yêu cầu API động được CloudFront chuyển tiếp đến Amazon API Gateway. API Gateway sử dụng Amazon Cognito để xác thực JWT Token của người dùng

2. **Điều hướng yêu cầu:** Sau khi xác thực thành công, API Gateway kích hoạt dịch vụ tương ứng với yêu cầu của người dùng gồm Lambda for History (truy vấn lịch sử) và AWS core Step Function (Chatbot) .
3. **Thực hiện yêu cầu:** Bên trong Step Functions Workflow, các hàm AWS Lambda được triệu gọi tuần tự:
   - Chatbot Lambda gọi Model LLM để phân tích ý định của người dùng và trích xuất Tool Call. Trạng thái chờ xác nhận được lưu vào bảng `PendingAction` trong Amazon DynamoDB.
   - Sau khi người dùng xác nhận, Dispatcher Lambda gọi AWS STS để thực hiện `AssumeRole` lấy quyền truy cập tạm thời. Sau đó, nó sử dụng AWS SDK (`Boto3`) để thực thi các lệnh quản lý tài nguyên (như EC2, S3, IAM) trên tài khoản AWS cá nhân của người dùng.
   - Formatter Lambda gọi lại Model LLM để chuyển đổi kết quả kỹ thuật thô thành ngôn ngữ tự nhiên, đồng thời ghi log hành động vào bảng `ActionHistory` trong DynamoDB và log hệ thống vào CloudWatch.
4. **Ghi nhận và trả kết quả:** Formatter Lambda publish thông báo real-time qua AWS AppSync. AppSync duy trì kết nối WebSocket persistent và đẩy thông báo trực tiếp về cho Client/Web

2.3 Đặc tả chi tiết cần thiết trong giải pháp của dự án

2.3.1 Đặc tả cấu hình dịch vụ Bảo mật và Điều hướng

1. Amazon Route 53

- **Tên miền & Subdomain:**
  - satibot.id.vn (Giao diện Web).
  - api.satibot.id.vn (API Gateway Endpoint).
- **Loại bản ghi:** Alias Record (trỏ tới CloudFront Distribution).
- **Chứng chỉ SSL/TLS:** AWS Certificate Manager (ACM).
  - Loại chứng chỉ: Wildcard (*.satibot.id.vn).
  - Region cấp phát: us-east-1.
- **Chính sách định tuyến:** Failover Routing.

- **Cấu hình bản ghi Primary:** Trỏ đến CloudFront Distribution (Region: ap-southeast-1).
- **Cấu hình bản ghi Secondary:** Trỏ đến CloudFront Distribution (Region: us-east-1; Origin: S3 Bucket chứa tệp error.html).
- **Cấu hình Health Check (áp dụng cho bản ghi Primary):**
  - Protocol: HTTPS.
  - Path: /health.
  - Request Interval: 10 giây.
  - Failure Threshold: 3 lần.
- **TTL:** 60 giây.

2. AWS WAF (Web Application Firewall)

- **Vị trí tích hợp:** Gắn vào CloudFront Distribution.
- **Cấu hình Web ACL:**
  - **Managed Rule Groups:**

    AWSManagedRulesCommonRuleSet: Action = Block.

    AWSManagedRulesSQLiRuleSet: Action = Block.
  - **Rate-based Rule:**

    Threshold: 100 requests / 5 phút / 1 IP.

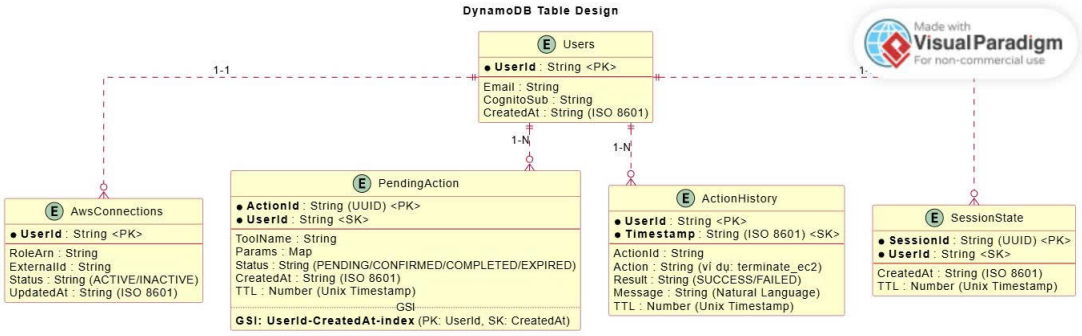
    Action: Block.
  - **IP Sets:** Không cấu hình.

2.3.2 Giao diện và giao thức tương tác giữa các components

Bảng 2. Đặc tả giao diện và giao thức tương tác giữa các components

Nguồn → Đích	Giao thức	Xác thực	Dữ liệu trao đổi
Web Client → CloudFront → S3	HTTPS	TLS	Tệp giao diện tĩnh
Web Client → API Gateway	HTTPS REST	Cognito JWT	Câu lệnh, session ID, mã hành động
API Gateway → Chatbot Lambda	Lambda Proxy	IAM	Nội dung chat và thông tin phiên
API Gateway → Step Functions	StartExecution	IAM Role	User ID và mã hành động đã xác nhận
Chatbot Lambda ↔ LLM	HTTPS API	Theo nhà cung cấp LLM	Prompt, schema công cụ, tham số và kết quả phân tích
Step Functions → Lambda Tasks	AWS SDK Integration	Step Functions Execution Role	JSON đầu vào/đầu ra từng bước
Lambda ↔ DynamoDB	Boto3 SDK	Lambda Execution Role	GetItem, PutItem, Query, UpdateItem
Dispatcher Lambda → AWS STS	Boto3 / HTTPS	Lambda Execution Role	AssumeRole với RoleArn và ExternalId
Dispatcher → Tài khoản AWS đích	Boto3 / HTTPS	Credential tạm thời, SigV4	Yêu cầu API và kết quả thao tác AWS
Formatter Lambda → AppSync	GraphQL Mutation / HTTPS	IAM SigV4	Nội dung kết quả cần phát tới frontend
Web Client ↔ AppSync	GraphQL Subscription / WSS	Cognito User Pool	Sự kiện cập nhật trên khung chat

2.3.3 Cơ sở dữ liệu hệ thống

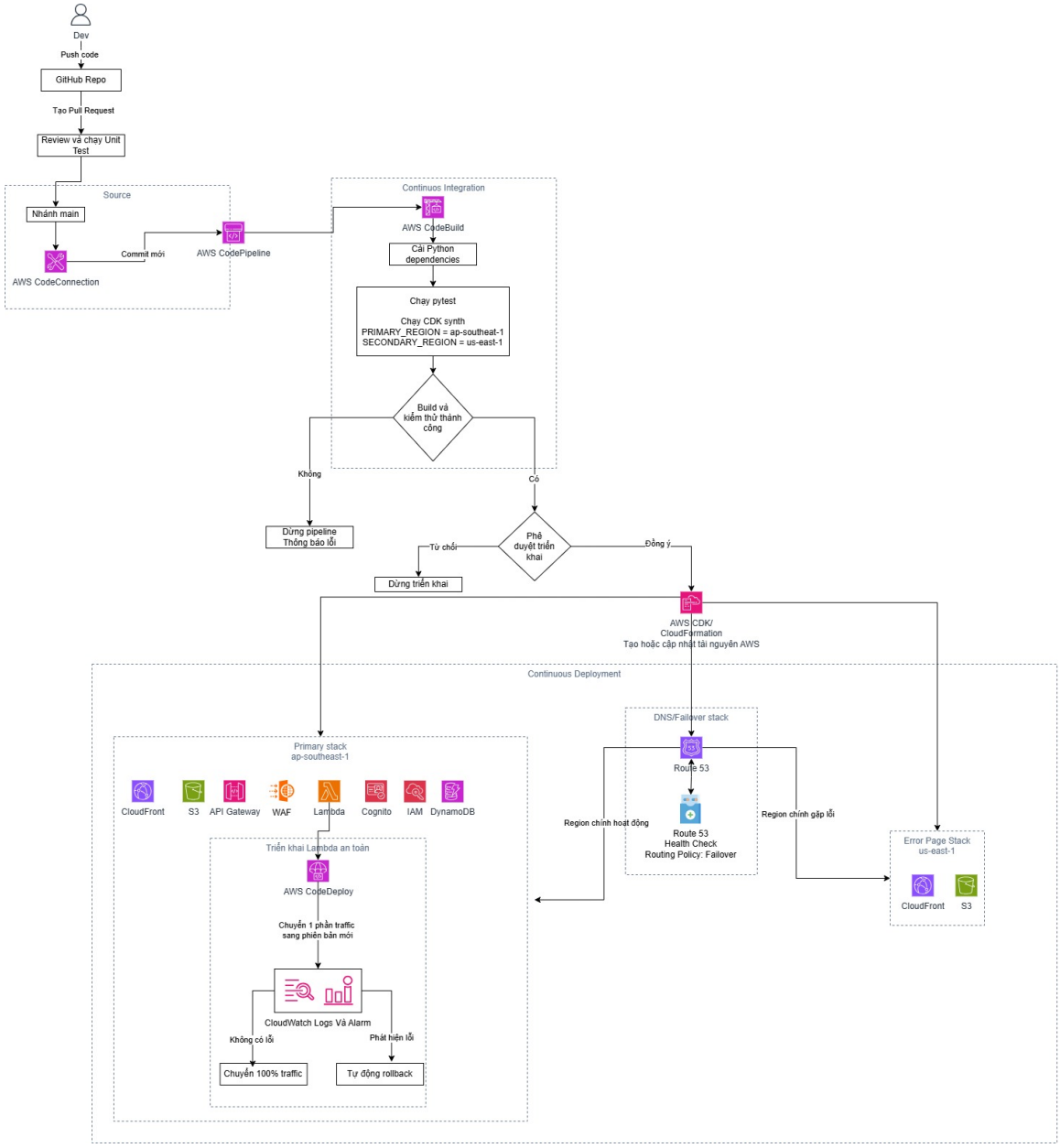


Hình 5. ERD cơ sở dữ liệu hệ thống

2.3.4 CI/CD

Quy trình CI/CD của SatiBot bắt đầu khi lập trình viên đẩy mã nguồn lên GitHub và tạo Pull Request. Mã nguồn được các thành viên trong nhóm xem xét, chạy kiểm thử đơn vị và chỉ được hợp nhất vào nhánh `main` khi đáp ứng yêu cầu. AWS CodeConnections theo dõi nhánh `main` và kích hoạt AWS CodePipeline khi xuất hiện commit mới. Trong giai đoạn tích hợp liên tục, AWS CodeBuild cài đặt Python cùng các thư viện cần thiết, chạy toàn bộ bộ kiểm thử bằng `pytest`, sau đó thực hiện `cdk synth` với hai biến cấu hình `PRIMARY_REGION=ap-southeast-1` và `SECONDARY_REGION=us-east-1`. Nếu quá trình build hoặc kiểm thử thất bại, pipeline dừng và phát thông báo lỗi; nếu thành công, quy trình chuyển sang bước phê duyệt thủ công. Khi yêu cầu triển khai được chấp thuận, AWS CDK và AWS CloudFormation tiến hành tạo mới hoặc cập nhật các tài nguyên AWS theo cấu hình đã khai báo.

Trong giai đoạn triển khai liên tục, hạ tầng được chia thành ba nhóm tài nguyên. Primary Stack tại `ap-southeast-1` chứa website chính và backend của SatiBot, bao gồm CloudFront, S3, API Gateway, WAF, Lambda, Cognito, IAM và DynamoDB. Các phiên bản Lambda mới được AWS CodeDeploy triển khai theo chiến lược canary; CloudWatch Logs và Alarm theo dõi trạng thái để chuyển toàn bộ lưu lượng sang phiên bản mới khi không có lỗi hoặc tự động rollback khi phát hiện sự cố. Error Page Stack tại `us-east-1` chỉ sử dụng S3 và CloudFront để cung cấp một trang thông báo lỗi độc lập, không cần đồng bộ dữ liệu ứng dụng với region chính. DNS/Failover Stack sử dụng Amazon Route 53 Health Check và chính sách Failover Routing để theo dõi hệ thống: khi region chính hoạt động bình thường, người dùng được điều hướng đến website tại `ap-southeast-1`; khi region này gặp sự cố, Route 53 chuyển kết quả phân giải DNS sang trang thông báo lỗi tại `us-east-1`.



Hình 6. Quy trình tích hợp, kiểm thử và triển khai liên tục của hệ thống SatiBot

2.3.5 Phân quyền cho các hàm Lambda

SatiBot phân quyền theo nhiệm vụ của từng Lambda. Mỗi hàm backend được gắn một IAM role để AWS xác định hàm được phép thực hiện thao tác nào và trên tài nguyên nào. Việc cấp quyền riêng giúp giới hạn tác động khi một hàm gặp lỗi hoặc xử lý sai yêu cầu.

Bảng 3 trình bày các quyền theo thiết kế cho năm Lambda backend và Lambda ứng dụng trong tài khoản người dùng. Các quyền đọc, ghi được giới hạn vào đúng bảng dữ liệu; quyền đọc khóa Gemini API chỉ được cấp cho những hàm sử dụng mô hình.

Bảng 3. Nhiệm vụ và quyền IAM của các Lambda

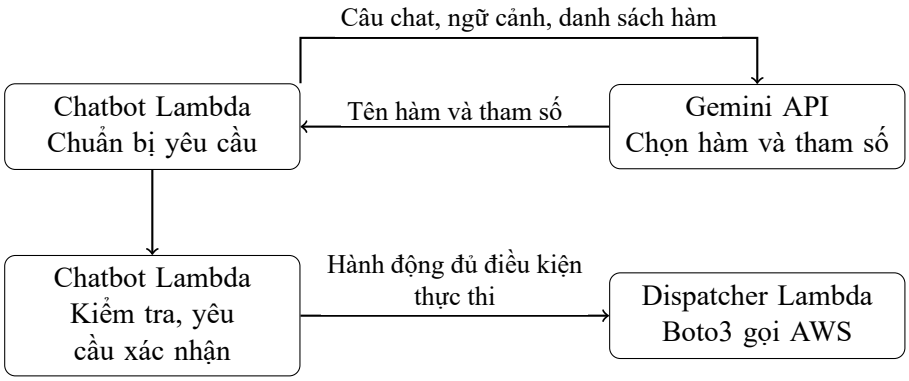
Lambda	**Nhiệm vụ**	**IAM Action cần cấp và nơi sử dụng**
Chatbot	Xử lý câu chat; lưu phiên và hành động chờ xác nhận.	`dynamodb:GetItem`: đọc SessionState, PendingAction và AwsConnections. `dynamodb:PutItem`, `dynamodb:UpdateItem`: ghi SessionState và PendingAction. `secretsmanager:GetSecretValue`: đọc khóa Gemini API.
Dispatcher	Thực thi hành động đã duyệt trên tài khoản AWS đích.	`dynamodb:GetItem`: đọc PendingAction và AwsConnections. `dynamodb:UpdateItem`: cập nhật PendingAction. `sts:AssumeRole`: nhận quyền tạm thời từ role trong tài khoản đích.
Formatter	Tạo lời đáp; lưu và gửi kết quả.	`dynamodb:PutItem`, `dynamodb:UpdateItem`: ghi ActionHistory. `appsync:GraphQL`: phát kết quả qua AppSync. `secretsmanager:GetSecretValue`: đọc khóa Gemini API.
Connector	Tạo ExternalId; lưu và kiểm tra kết nối AWS.	`dynamodb:GetItem`, `dynamodb:PutItem`, `dynamodb:UpdateItem`: quản lý AwsConnections. `sts:AssumeRole`: kiểm tra role người dùng cung cấp.
History	Liệt kê và xem chi tiết lịch sử.	`dynamodb:Query`: liệt kê bản ghi trong ActionHistory. `dynamodb:GetItem`: đọc một bản ghi trong ActionHistory.
Lambda của người dùng	Chạy mã ứng dụng trong tài khoản AWS đích.	Quyền phụ thuộc mã ứng dụng; ví dụ `s3:GetObject` khi hàm cần đọc tệp S3. Quyền quản lý hàm, như `lambda:InvokeFunction` hoặc `lambda:UpdateFunctionConfiguration`, nằm trong role đích mà Dispatcher đảm nhận.

Để thao tác trên tài khoản AWS của người dùng, Connector và Dispatcher sử dụng role do chủ tài khoản cho phép SatiBot truy cập. Connector kiểm tra kết nối; Dispatcher nhận quyền tạm thời qua STS để thực hiện các thao tác được role đó cho phép. Lambda ứng dụng của người dùng có quyền chạy riêng, tùy theo mã ứng dụng. Các Lambda ghi

log được cấp thêm quyền ghi CloudWatch Logs để theo dõi lỗi.

2.3.6 Tích hợp LLM qua Gemini API

Hệ thống sử dụng mô hình Gemini của Google qua Gemini API. Tại Chatbot Lambda, mô hình chuyển câu chat tiếng Việt thành yêu cầu gọi hàm quản trị AWS; tại Formatter Lambda, mô hình diễn giải kết quả thực thi. Việc thao tác tài nguyên do Dispatcher thực hiện bằng Boto3 theo luồng ở Hình 3.



Hình 7. Tích hợp Gemini API với các Lambda xử lý yêu cầu

Backend gọi Gemini API bằng HTTP POST qua HTTPS, sử dụng endpoint Chat Completions do Google cung cấp. Nội dung request và response có định dạng JSON; khóa API được gửi trong header Authorization. Endpoint và mã model là cấu hình backend, model cần hỗ trợ Function Calling. Theo phương án triển khai, khóa API được lưu trong Secrets Manager và chỉ các Lambda cần gọi LLM được quyền đọc.

Bảng 4. Dữ liệu trao đổi giữa Chatbot Lambda và Gemini API

Chiều trao đổi	**Trường dữ liệu**	**Nội dung**
Gửi tới Gemini	`model`	Mã mô hình Gemini được cấu hình cho hệ thống.
	`messages`	Chỉ dẫn hệ thống, câu chat hiện tại và ngữ cảnh hội thoại cần thiết.
	`tools`	Danh sách hàm được phép gọi: tên, mô tả và JSON Schema quy định tham số.
	`tool_choice`	Giá trị `auto`: cho phép mô hình trả lời hoặc đề xuất gọi hàm theo yêu cầu.
Nhận từ Gemini	`content`	Nội dung trả lời hoặc câu hỏi bổ sung khi chưa đủ thông tin.
	`tool_calls`	Danh sách lời gọi hàm; mỗi lời gọi có tên hàm và các tham số.

Danh sách hàm được khai báo trước cho bốn module EC2, S3, IAM và Lambda. Mỗi khai báo mô tả một thao tác và các tham số bắt buộc, kiểu dữ liệu, giá trị cho phép. Khi nhận `tool_calls`, Chatbot kiểm tra tên hàm và tham số theo khai báo, rồi ánh xạ sang thao tác tương ứng của Dispatcher.

Formatter gửi kết quả AWS cho LLM để tạo lời đáp dễ đọc, không cần cung cấp danh sách hàm quản trị. Trạng thái thực thi, định danh tài nguyên và số liệu được lấy từ kết quả AWS; nội dung do LLM diễn giải không thay thế các dữ liệu này. Nếu bước diễn giải lỗi, Formatter trả kết quả theo mẫu có sẵn.

Mỗi lần gọi LLM có thời gian chờ và số lần thử lại giới hạn. Phản hồi sai cấu trúc không được chuyển sang thực thi. Ngữ cảnh hội thoại được giới hạn số lượt và dung lượng; credential AWS không được gửi tới mô hình. Lỗi kết nối hoặc lỗi tạo lời đáp không làm chạy lại thao tác thay đổi tài nguyên đã thực hiện.

2.3.7 Endpoint tiếp nhận hội thoại và trích xuất ý định (POST /chat)

1. Thông tin chung:

- **Đường dẫn (Path):** `/chat` (hoặc `/api/chat` trên giao diện Web Client).
- **Phương thức HTTP:** `POST`.
- **Cơ chế xác thực:** Bearer JWT Token thông qua HTTP Header `Authorization: Bearer <Token>`. Amazon API Gateway sử dụng Cognito User Pool Authorizer để giải mã và kiểm tra tính hợp lệ của token trước khi kích hoạt hàm xử lý phía sau.
- **Thành phần tiếp nhận:** Amazon API Gateway → Chatbot Lambda (tích hợp mô hình ngôn ngữ lớn LLM với cơ chế Function Calling).

2. Dữ liệu trao đổi thực tế (Kịch bản khởi tạo máy chủ ảo EC2):

Mẫu JSON Request gửi từ Web Client khi người dùng gửi câu lệnh khởi tạo máy chủ EC2 bằng ngôn ngữ tự nhiên tiếng Việt:

```
{
  "message": "Tạo cho tôi một máy chủ EC2 Ubuntu có tên 'web-demo'",
  "history": [
    {
      "role": "user",
      "content": "Xin chào SatiBot"
    },
    {
      "role": "assistant",
      "content": "Xin chào! Tôi là SatiBot - Trợ lý ChatOps quản trị AWS."
    }
  ],
  "region": "ap-southeast-2"
}
```

Mẫu JSON Response trả về từ hệ thống. Do thao tác `create_instance` là hành động làm thay đổi tài nguyên hạ tầng và phát sinh chi phí, hệ thống trả về mã trạng thái `CONFIRMATION_REQUIRED` kèm mã giao dịch `action_id` để yêu cầu xác nhận:

```
{
  "status": "CONFIRMATION_REQUIRED",
  "type": "action_confirmation",
  "action_id": "act_8f3a9e1b2c4d",
  "service": "ec2",
  "tool_name": "create_instance",
  "parameters": {
    "instance_name": "web-demo",
    "os_type": "ubuntu",
    "instance_type": "t3.micro"
  },
  "message": "Thao tác 'create_instance' làm thay đổi cấu hình hoặc kích hoạt
            " tài nguyên. Bạn có chắc chắn muốn thực hiện không?"
}
```

3. Bảng giải thích chi tiết các trường dữ liệu:

Bảng 5. Mô tả các trường trong Request Body của Endpoint `POST /chat`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`message`	String	Có	Nội dung câu lệnh bằng ngôn ngữ tự nhiên tiếng Việt của người dùng gửi từ giao diện.
`history`	Array[Object]	Không	Lịch sử hội thoại nhiều lượt gồm các cặp `role` và `content`, đồng bộ từ bảng `SessionState` trong DynamoDB.
`region`	String	Không	Khu vực AWS mục tiêu thực hiện tác vụ (mặc định: `ap-southeast-2`).
`credentials`	Object	Không	Thông tin xác thực AWS cục bộ.

Bảng 6. Mô tả các trường trong Response Body của Endpoint `POST /chat`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`status`	String	Có	Trạng thái phản hồi của hệ thống (`CONFIRMATION_REQUIRED` khi cần duyệt, hoặc `SUCCESS` khi chỉ truy vấn dữ liệu).
`type`	String	Có	Loại phản hồi định hướng hiển thị phía Client (`action_confirmation` hoặc `chat_reply`).
`action_id`	String	Có	Mã định danh duy nhất cho hành động chờ duyệt, được lưu làm Partition Key trong bảng `PendingAction` với thời gian TTL 15 phút.
`service`	String	Có	Dịch vụ AWS mục tiêu (ví dụ: `ec2`, `s3`, `lambda`, `iam`) do LLM Function Calling trích xuất.
`tool_name`	String	Có	Tên hành động kỹ thuật được trích xuất (ví dụ: `create_instance`, `manage_power`).
`parameters`	Object	Có	Cấu trúc tham số kỹ thuật chi tiết trích xuất từ câu lệnh tự nhiên của người dùng.
`message`	String	Có	Lời nhắn tiếng Việt thân thiện gửi người dùng để thông báo hoặc yêu cầu xác nhận an toàn.

2.3.8 Endpoint xác nhận và thực thi tài nguyên (POST /confirm)

1. Thông tin chung:

- **Đường dẫn (Path):** `/confirm` (hoặc gửi tới `/chat` với tham số `confirmed: true`).
- **Phương thức HTTP:** `POST`.
- **Cơ chế xác thực:** Bearer JWT Token của Amazon Cognito trong Header `Authorization`.
- **Thành phần tiếp nhận:** Amazon API Gateway → Chatbot Lambda / Step Functions → Dispatcher Lambda → EC2 Service (`boto3.client('ec2')`) → Amazon DynamoDB (`ActionHistory`).

2. Dữ liệu trao đổi thực tế (Kịch bản người dùng chấp thuận tạo máy ảo):

Mẫu JSON Request gửi từ Web Client khi người dùng nhấn nút "Xác nhận thực thi" trên giao diện:

```
{
  "confirmed": true,
  "action_id": "act_8f3a9e1b2c4d",
  "service": "ec2",
  "tool_name": "create_instance",
  "parameters": {
    "instance_name": "web-demo",
    "os_type": "ubuntu",
    "instance_type": "t3.micro"
```

```
  },
  "region": "ap-southeast-2"
}
```

Mẫu JSON Response trả về ngay lập tức sau khi Dispatcher Lambda và Module EC2 Service thực thi gọi Boto3 SDK thành công trên AWS Sydney:

```
{
  "status": "SUCCESS",
  "type": "action_executed",
  "service": "ec2",
  "tool_name": "create_instance",
  "message": "Đã gửi yêu cầu khởi tạo máy chủ 'web-demo' (i-015aad95042137744
             " thành công với trạng thái 'pending'.",
  "data": {
    "instance_id": "i-015aad95042137744",
    "instance_name": "web-demo",
    "state": "pending",
    "instance_type": "t3.micro",
    "image_id": "ami-0af9110f0ec0820e2"
  }
}
```

3. Bảng giải thích chi tiết các trường dữ liệu:

Bảng 7. Mô tả các trường trong Request Body của Endpoint `POST /confirm`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`confirmed`	Boolean	Có	Cờ xác nhận mang giá trị `true`, khẳng định người dùng đã trực tiếp phê duyệt hành động.
`action_id`	String	Có	Mã hành động tương ứng đã tạo ở bước `/chat`, dùng để đối soát trạng thái trong bảng `PendingAction`.
`service`	String	Có	Dịch vụ AWS đích cần điều phối thực thi (`ec2`, `s3`, `lambda`, `iam`).
`tool_name`	String	Có	Tên hành động kỹ thuật cần thực thi (`create_instance`, v.v.).
`parameters`	Object	Có	Tập tham số kỹ thuật đã qua rà soát của người quản trị.
`region`	String	Không	Khu vực thực thi lệnh trên đám mây AWS (mặc định: `ap-southeast-2`).

Bảng 8. Mô tả các trường trong Response Body của Endpoint `POST /confirm`

Tên trường	Kiểu dữ liệu	Bắt buộc	Mô tả ý nghĩa và nguồn gốc dữ liệu
`status`	String	Có	Kết quả xử lý hạ tầng tổng thể (`SUCCESS` hoặc `ERROR`).
`type`	String	Có	Phân loại giao diện (`action_executed`) để Web Client cập nhật trạng thái thẻ hiển thị sang hoàn tất.
`service`	String	Có	Tên dịch vụ AWS đã thực thi hoàn tất (`ec2`).
`tool_name`	String	Có	Tên công cụ kỹ thuật đã chạy (`create_instance`).
`message`	String	Có	Thông báo tổng kết kỹ thuật bằng tiếng Việt kèm mã ID và trạng thái máy ảo.
`data`	Object	Có	Đối tượng chứa dữ liệu chi tiết trả về từ Boto3 SDK gồm: `instance_id`, `instance_name`, `state` (trạng thái `pending` non-blocking), `instance_type` và mã `image_id` chính thức của hệ điều hành.

3. KẾ HOẠCH TRIỂN KHAI DỰ ÁN

3.1 Kế hoạch triển khai theo từng giai đoạn (05 tuần)

Dự án được lên kế hoạch thực hiện trong 5 tuần theo mô hình Agile/Scrum, chia thành 5 Sprint tương ứng:

- **Tuần 1 (Sprint 1) – Chuẩn bị môi trường và kiểm thử PoC độc lập:** Thống nhất quy định làm việc, quy chuẩn viết code chung. Tạo tài khoản AWS chung và IAM User cho từng thành viên. Mỗi thành viên tìm hiểu tài liệu kỹ thuật, viết và kiểm thử độc lập các hàm Python/Boto3 cơ bản cho dịch vụ AWS mình phụ trách (S3, EC2, Lambda, IAM).
- **Tuần 2 (Sprint 2) – Xây dựng hạ tầng cơ bản và ghép nối Backend:** Dựng khung giao diện web React tĩnh trên S3/CloudFront; cấu hình đăng ký, đăng nhập tài khoản qua Amazon Cognito. Thiết lập API Gateway, viết Lambda Backend tiếp nhận yêu cầu, tạo 05 bảng trên DynamoDB và tích hợp các module Boto3 của 4 thành viên vào Backend.
- **Tuần 3 (Sprint 3) – Tích hợp AI Chatbot, Step Functions và AppSync:** Tích hợp mô hình ngôn ngữ lớn (LLM), viết prompt và khai báo cấu trúc Function Calling (Tool Schemas). Dựng máy trạng thái Step Functions để điều phối luồng: nhận câu chat $\rightarrow$ Backend gọi LLM trích xuất tham số $\rightarrow$ hiển thị xác nhận $\rightarrow$ xác thực qua STS Assume-Role $\rightarrow$ Boto3 thực thi trên AWS $\rightarrow$ trả kết quả lên giao diện qua kênh WebSocket của AWS AppSync.
- **Tuần 4 (Sprint 4) – Kiểm thử hệ thống và tối ưu hóa:** Kiểm thử các kịch bản thực tế, bao gồm cả trường hợp chạy thành công và các tình huống thiếu quyền (AccessDenied); cấu hình CloudWatch Logs để theo dõi lỗi tập trung; tối ưu thời gian phản hồi của Lambda và hoàn thiện giao diện chat.

- **Tuần 5 (Sprint 5) – Nghiệm thu, dọn dẹp tài nguyên và hoàn thiện báo cáo:** Chạy thử toàn bộ các ca sử dụng để nghiệm thu sản phẩm; hoàn thiện báo cáo kỹ thuật, slide thuyết trình; rà soát và xóa các tài nguyên AWS thử nghiệm để tránh phát sinh chi phí.

3.2 Kế hoạch hành động chi tiết theo từng Sprint

Bảng 9. Kế hoạch hành động và phân công công việc chi tiết theo từng Sprint

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Người phụ trách	Kết quả bàn giao
Giai đoạn 1: Tuần 1 – Thiết lập quy trình, hạ tầng ban đầu và PoC Boto3 độc lập				
1	Viết và kiểm thử độc lập module Boto3 cho dịch vụ S3	User's Cloud (Amazon S3)	Phạm Đức Hùng	Module `s3_service` (tạo bucket, presign, block public).
2	Thống nhất kịch bản và kiểm thử module Boto3 cho dịch vụ EC2	User's Cloud (Amazon EC2)	Nguyễn Đức Huy	Module `ec2_service` (khởi tạo, bật/tắt, describe).
3	Thống nhất kịch bản và kiểm thử module Boto3 cho Lambda	User's Cloud (AWS Lambda)	Trịnh Thiên Quang	Module `lambda_service` (list, invoke, hot config).
4	Thống nhất quy tắc làm việc, tạo tài khoản AWS và module IAM	Toàn bộ kiến trúc, AWS IAM	Cao Anh Minh	Quy chuẩn mã nguồn, AWS Account chung, module `iam_service`.
Giai đoạn 2: Tuần 2 – Dựng hạ tầng nền tảng Serverless, Giao diện và Tích hợp Backend				
5	Phát triển khung giao diện React cơ bản trên S3/CloudFront	Frontend (React, S3, CloudFront)	Phạm Đức Hùng	Giao diện Web Chat responsive, CDN CloudFront với HTTPS.
6	Cấu hình luồng đăng nhập và quản lý danh tính qua Cognito	Amazon Cognito, API Gateway	Nguyễn Đức Huy	Luồng Login/Register qua Cognito, JWT Authorizer trên Gateway.
7	Dựng Lambda Backend trung tâm và tích hợp các module Boto3	AWS Lambda Backend	Trịnh Thiên Quang	Lambda Backend tiếp nhận request và tích hợp các module Boto3.
8	Cấu hình IAM Role cho Backend và thiết kế 05 bảng DynamoDB	AWS IAM, DynamoDB	Cao Anh Minh	IAM Execution Role, 05 bảng DynamoDB cơ sở.
Giai đoạn 3: Tuần 3 – Tích hợp AI Chatbot, Step Functions và AppSync				
9	Định nghĩa Function Calling và Prompt cho dịch vụ S3	Model LLM, Chatbot Lambda	Phạm Đức Hùng	Bộ Tool Schema cho S3, hỗ trợ kênh WebSocket AppSync.
10	Định nghĩa Function Calling cho EC2 và ghép luồng LLM	Model LLM, Chatbot Lambda	Nguyễn Đức Huy	Bộ Tool Schema cho EC2, hoàn thiện ghép luồng LLM.
11	Tích hợp API LLM, xử lý phản hồi Function Calling tại Backend	Model LLM, Lambda Backend	Trịnh Thiên Quang	Xử lý phản hồi Function Calling và tích hợp LLM vào Backend.
12	Hoàn thiện Prompt IAM, xây dựng Step Functions State Machine	AWS Step Functions, STS	Cao Anh Minh	State Machine điều phối luồng thực thi, xác thực STS.
Giai đoạn 4: Tuần 4 – Kiểm thử toàn diện, Phân quyền và Tối ưu hóa				
13	Kiểm thử hệ thống, tối ưu hóa module S3	Phân hệ S3, Web Client	Phạm Đức Hùng	Báo cáo kiểm thử S3, tối ưu giao diện truyền tải file.
14	Kiểm thử hệ thống, tối ưu module EC2, dọn dẹp tài nguyên	Phân hệ EC2, CloudWatch	Nguyễn Đức Huy	Báo cáo kiểm thử EC2, kịch bản FinOps cảnh báo máy ảo.
15	Kiểm thử phân quyền, tối ưu hệ thống Backend	Backend Lambda, Step Functions	Trịnh Thiên Quang	Báo cáo kiểm thử phân quyền, tối ưu Cold-start của Lambda.
16	Kiểm thử trường hợp thiếu quyền (AccessDenied), CloudWatch Log	AWS IAM, CloudWatch Logs	Cao Anh Minh	Xử lý lỗi phân quyền, tập trung hóa log trên CloudWatch.
Giai đoạn 5: Tuần 5 – Nghiệm thu, Tổng kết dự án và Đóng gói báo cáo				
17	Hoàn thiện báo cáo phần S3, hỗ trợ làm slide thuyết trình	Báo cáo đề tài	Phạm Đức Hùng	Nội dung báo cáo S3, hoàn thiện slide demo.
18	Tổng hợp kết quả nghiệm thu EC2, chuẩn bị slide demo	Báo cáo đề tài	Nguyễn Đức Huy	Nội dung báo cáo EC2, slide thuyết trình kỹ thuật.

Bảng 9. (Tiếp theo Bảng 9)

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Người phụ trách	Kết quả bàn giao
19	Nghiệm thu luồng Lambda, hoàn thiện tài liệu hướng dẫn	Báo cáo đề tài	Trịnh Thiên Quang	Nội dung báo cáo Lambda, tài liệu vận hành.
20	Tổng kết dự án, hoàn thiện báo cáo tổng thể, slide thuyết trình	Báo cáo đề tài	Cao Anh Minh	Bản báo cáo hoàn chỉnh (`report1.pdf`), slide tổng thể.